

NIT2112 Object-Oriented Programming

Practical Coding Test: EventPro Conference Management System

Assessment Type	Open-Book Practical Coding Test
Duration	1 Hour
Total Marks	100
Weighting	[Insert %] of Unit Total
AI Copilot Use	REQUIRED (Must be documented)

Important Instructions

1. Read the entire specification before beginning to code
2. You MUST use an AI Copilot throughout this test this is assessed
3. Document your AI interactions as you work (copy key prompts/responses)
4. Save your work frequently
5. Submit both your Python file AND reflection document

Learning Outcomes Assessed

LO	Description
LO 2	Utilise AI Copilots for OOP Develop
LO 3	Solve Practical Problems Using OOP
LO 4	Optimize OOP Systems with AI: Apply

Scenario: EventPro Conference Management

You have been hired by EventPro Solutions, a company that organizes professional conferences, workshops, and networking events. They need a Python system to manage events, handle attendee registrations, process different ticket types, and coordinate session scheduling.

The Event Environment

EventPro manages:

- Events: Conferences, workshops, and webinars with different characteristics
- Attendees: Participants with various registration types
- Tickets: Different ticket tiers with varying benefits
- Sessions: Individual talks or workshops within events
- Speakers: Presenters who deliver sessions

Your Task

Design and implement an Object-Oriented event management system that models this environment, enforces business rules, and demonstrates your understanding of OOP principles.

Functional Requirements

FR1: Event Hierarchy

Create an event class hierarchy to represent different types of events:

Event Type	Additional Attributes	Special Rules
Event (Base)	event_id, name, date, capacity	Abstract cannot be instantiated di
Conference	num_tracks, keynote_speaker	Multi-track; requires keynote; min
Workshop	instructor, skill_level, materials_	Max 30 participants; hands-on forma
Webinar	platform, recording_available, time	Unlimited capacity (set capacity to

All events must:

- Have a unique event_id
- Support calculate_revenue() method returning total ticket sales
- Implement is_registration_open() method (polymorphic behaviour)
- Provide display_details() method showing event information
- Track registered attendees

FR2: Ticket System

Implement a ticket hierarchy with the following:

Ticket Type	Price Modifier	Benefits
Ticket (Base)	Base price	Basic access only
StandardTicket	1.0	Access to main sessions
VIPTicket	2.5	Priority seating, networking dinner
EarlyBirdTicket	0.7	30% discount, only available 30+ da
GroupTicket	0.8 per person	Requires minimum 5 attendees, 20% d

Ticket validation rules:

- EarlyBirdTicket can only be purchased if event date is > 30 days away
- GroupTicket must specify group_size 5
- VIPTicket has limited availability (max 20% of event capacity)

FR3: Attendee Registration

Implement attendee management with these rules:

Requirement	Description
-------------	-------------

Registration	Attendees have attendee_id, name, e
Capacity Check	Cannot register if event is full
Duplicate Check	Same email cannot register twice fo
Waitlist	If event full, attendee can join wa

Attendee Types:

- Attendee (Base): Standard participant
- SpeakerAttendee: Attendee who is also presenting (complimentary VIP ticket)
- SponsorAttendee: Attendee from sponsor company (special badge, booth access)

FR4: Session Scheduling

Implement session management:

Rule	Description
Session Creation	Sessions have session_id, title, sp
Time Slot	Sessions must have start time and c
Speaker Conflict	A speaker cannot present two sessio
Capacity	Room capacity must accommodate expe

FR5: Waitlist Notifications (Observer Pattern)

Implement a notification system:

- When a registered attendee cancels, notify the first person on the waitlist
- WaitlistManager observes registration changes and processes the queue
- Notifications should include event name and available spot details
- FIFO (First-In-First-Out) processing for fairness

FR6: Event Factory (Factory Pattern)

Implement a Factory Pattern to create events:

```
factory = EventFactory()
conf = factory.create_event("conference", event_id="CONF001", name="PyCon AU 2025",
                             date="2025-08-15", capacity=500, num_tracks=4,
                             keynote_speaker="Guido van Rossum")
workshop = factory.create_event("workshop", event_id="WS001", name="Django Masterclass",
                                 date="2025-07-20", capacity=25, instructor="Sarah Chen",
                                 skill_level="Intermediate", materials_included=True)
```

FR7: Legacy Ticketing Integration (Adapter Pattern)

EventPro has a legacy ticketing system from a previous vendor:

```
class LegacyTicketingSystem:
    def __init__(self):
        self.old_bookings = {
            "BK001": {"guest_name": "John Smith", "email_addr": "john@email.com",
```

```

        "event_code": "EVT100", "tier": "GOLD", "amount_paid": 15000},
        "BK002": {"guest_name": "Jane Doe", "email_addr": "jane@email.com",
        "event_code": "EVT100", "tier": "SILVER", "amount_paid": 8000},
    }

```

```

def fetch_booking(self, booking_ref):
    return self.old_bookings.get(booking_ref, None)

```

Create an Adapter that converts legacy data into your system's format:

- amount_paid ticket_price (convert cents to dollars)
- tier GOLD VIPTicket, SILVER StandardTicket
- guest_name name
- email_addr email

FR8: Event Registry (Singleton Pattern)

Ensure only one EventRegistry exists to maintain consistency across the application.

Technical Requirements

Your solution MUST demonstrate the following OOP concepts:

Concept	Requirement	Marks
Encapsulation	Use private/protected attributes wi	10
Inheritance	Create meaningful class hierarchies	10
Polymorphism	Override methods to provide type-sp	10
Abstraction	Use ABC with at least 2 abstract me	10
Interfaces	Define at least 1 interface (e.g.,	5
Custom Exceptions	Create exception hierarchy for erro	10
Factory Pattern	Implement event creation via factor	10
Adapter Pattern	Integrate legacy system using adapt	5
Observer Pattern	Implement waitlist notifications	10
Singleton Pattern	Ensure only one EventRegistry exist	5

Custom Exceptions Required

Implement the following exception hierarchy:

```

EventManagerError (base)
├── RegistrationError
│   ├── EventFullError
│   ├── DuplicateRegistrationError
│   └── InvalidTicketError
├── SchedulingError
│   ├── RoomConflictError
│   └── SpeakerConflictError
└── EventError

```

Use these exceptions appropriately throughout your code with meaningful error messages.

AI Copilot Usage Requirements

You MUST use your AI Copilot for:

1. Design Exploration Ask about OOP design options and trade-offs
2. Code Generation Generate boilerplate or complex implementations
3. Refactoring Improve code structure and OOP adherence
4. Documentation Generate docstrings and comments
5. Debugging Identify and fix logical errors

Good AI Copilot Usage Examples

Prompt Type	Example
Design	"Should I use composition or inheri
Implementation	"Help me implement the Observer pat
Refactoring	"How can I refactor this ticket val
Debugging	"Why is my EarlyBirdTicket validati

Poor AI Copilot Usage Examples

Issue	Example
Wholesale delegation	"Write the entire event management
No critical thinking	Accepting generated code without un
No documentation	Using AI but not recording how it i
Ignoring context	Pasting AI code that doesn't integr

Starter Code Template

```
"""
NIT2112 Practical Test: EventPro Conference Management System
Student Name: [Your Name]
Student ID: [Your ID]
Date: [Date]

AI Copilot Used: [Yes/No - Name of tool]
"""

from abc import ABC, abstractmethod
from datetime import date, datetime, timedelta
from typing import List, Optional, Dict
```

```

# =====
# CUSTOM EXCEPTIONS
# =====

class EventManagementError(Exception):
    """Base exception for event management system errors."""
    pass

class RegistrationError(EventManagementError):
    """Base exception for registration-related errors."""
    pass

class EventFullError(RegistrationError):
    """Raised when attempting to register for a full event."""
    pass

# TODO: Implement remaining exception hierarchy:
# - DuplicateRegistrationError
# - InvalidTicketError
# - SchedulingError
# - RoomConflictError
# - SpeakerConflictError
# - EventError
# - InvalidEventConfigError

# =====
# INTERFACES / ABSTRACT BASE CLASSES
# =====

class Registrable(ABC):
    """Interface for entities that support registration."""

    @abstractmethod
    def register_attendee(self, attendee: 'Attendee') -> bool:
        """Register an attendee. Returns True if successful."""
        pass

    @abstractmethod
    def get_available_spots(self) -> int:
        """Return number of available registration spots."""
        pass

class Schedulable(ABC):
    """Interface for entities that can be scheduled."""

    @abstractmethod
    def get_time_slot(self) -> tuple:
        """Return (start_time, end_time) tuple."""
        pass

```

```

    @abstractmethod
    def conflicts_with(self, other: 'Schedulable') -> bool:
        """Check if this schedule conflicts with another."""
        pass

# =====
# OBSERVER PATTERN - Waitlist Notifications
# =====
class WaitlistObserver(ABC):
    """Abstract observer for waitlist notifications."""

    @abstractmethod
    def notify(self, event_name: str, message: str) -> None:
        """Called when a spot becomes available."""
        pass

class WaitlistManager:
    """
    Manages the waitlist queue and notifications.
    Implements FIFO processing.
    """

    def __init__(self):
        # TODO: Initialize waitlist queue
        pass

    def add_to_waitlist(self, attendee: 'Attendee', event: 'Event') -> None:
        """Add an attendee to the waitlist for an event."""
        # TODO: Implement
        pass

    def process_cancellation(self, event: 'Event') -> Optional['Attendee']:
        """Process a cancellation - return next waitlisted attendee if any."""
        # TODO: Implement FIFO processing
        pass

# =====
# TICKET HIERARCHY
# =====
class Ticket(ABC):
    """
    Abstract base class for all ticket types.

    Attributes:
        ticket_id (str): Unique ticket identifier
        base_price (float): Base price before modifiers
        event (Event): The event this ticket is for
    """

    def __init__(self, ticket_id: str, base_price: float, event: 'Event'):

```

```

        # TODO: Implement with proper encapsulation
        pass

    @abstractmethod
    def calculate_price(self) -> float:
        """Calculate final ticket price with any modifiers."""
        pass

    @abstractmethod
    def get_benefits(self) -> List[str]:
        """Return list of benefits included with this ticket."""
        pass

    def is_valid(self) -> bool:
        """Check if ticket is valid for purchase."""
        # TODO: Base validation - override in subclasses for specific rules
        pass


class StandardTicket(Ticket):
    """Standard admission ticket."""

    PRICE_MODIFIER = 1.0

    def calculate_price(self) -> float:
        # TODO: Implement
        pass

    def get_benefits(self) -> List[str]:
        # TODO: Return ["Access to main sessions"]
        pass


class VIPTicket(Ticket):
    """
    VIP ticket with premium benefits.
    Limited to 20% of event capacity.
    """

    PRICE_MODIFIER = 2.5
    MAX_PERCENTAGE = 0.20

    def calculate_price(self) -> float:
        # TODO: Implement
        pass

    def get_benefits(self) -> List[str]:
        # TODO: Return VIP benefits list
        pass

    def is_valid(self) -> bool:
        # TODO: Check VIP availability limit
        pass

```



```

class EarlyBirdTicket(Ticket):
    """
    Discounted ticket for early registrations.
    Only available 30+ days before event.
    """

    PRICE_MODIFIER = 0.7
    MIN_DAYS_BEFORE = 30

    def calculate_price(self) -> float:
        # TODO: Implement 30% discount
        pass

    def get_benefits(self) -> List[str]:
        # TODO: Return early bird benefits
        pass

    def is_valid(self) -> bool:
        # TODO: Check if 30+ days before event
        pass

class GroupTicket(Ticket):
    """
    Group booking ticket with volume discount.
    Requires minimum 5 attendees.
    """

    PRICE_MODIFIER = 0.8
    MIN_GROUP_SIZE = 5

    def __init__(self, ticket_id: str, base_price: float, event: 'Event', group_size: int):
        # TODO: Implement with group_size validation
        pass

    def calculate_price(self) -> float:
        # TODO: Return price × group_size × modifier
        pass

    def get_benefits(self) -> List[str]:
        # TODO: Return group benefits
        pass

    def is_valid(self) -> bool:
        # TODO: Validate minimum group size
        pass

# =====
# ATTENDEE HIERARCHY
# =====

```

```

class Attendee(WaitlistObserver):
    """
    Base class for event attendees.

    Attributes:
        attendee_id (str): Unique identifier
        name (str): Full name
        email (str): Contact email
        ticket (Ticket): Assigned ticket
    """

    def __init__(self, attendee_id: str, name: str, email: str):
        # TODO: Implement with encapsulation
        pass

    def assign_ticket(self, ticket: Ticket) -> None:
        """Assign a ticket to this attendee."""
        # TODO: Implement
        pass

    def notify(self, event_name: str, message: str) -> None:
        """Receive notification about waitlist status."""
        # TODO: Implement notification handling
        pass

    def display_info(self) -> None:
        """Display attendee information."""
        # TODO: Implement
        pass


class SpeakerAttendee(Attendee):
    """
    Attendee who is also a speaker at the event.
    Receives complimentary VIP ticket.
    """

    def __init__(self, attendee_id: str, name: str, email: str,
                  bio: str, session_title: str):
        # TODO: Implement
        pass

    def display_info(self) -> None:
        # TODO: Override to show speaker details
        pass


class SponsorAttendee(Attendee):
    """
    Attendee from a sponsor company.
    Has booth access and special badge.
    """

```

```

def __init__(self, attendee_id: str, name: str, email: str,
              company_name: str, sponsorship_tier: str):
    # TODO: Implement
    pass

def display_info(self) -> None:
    # TODO: Override to show sponsor details
    pass

# =====
# EVENT HIERARCHY
# =====

class Event(Registrable, ABC):
    """
    Abstract base class for all events.

    Attributes:
        event_id (str): Unique identifier
        name (str): Event name
        date (date): Event date
        capacity (int): Maximum attendees
        attendees (list): Registered attendees
        base_ticket_price (float): Base price for tickets
    """

    def __init__(self, event_id: str, name: str, event_date: date,
                  capacity: int, base_ticket_price: float = 100.0):
        # TODO: Implement with proper encapsulation
        self._observers: List[WaitlistObserver] = []
        pass

    @abstractmethod
    def display_details(self) -> None:
        """Display event details. Must be overridden."""
        pass

    @abstractmethod
    def is_registration_open(self) -> bool:
        """Check if registration is still open."""
        pass

    def register_attendee(self, attendee: Attendee) -> bool:
        """
        Register an attendee for this event.

        Raises:
            EventFullError: If event is at capacity
            DuplicateRegistrationError: If email already registered
        """
        # TODO: Implement with validation
        pass

```

```
def cancel_registration(self, attendee_id: str) -> None:
    """
    Cancel an attendee's registration.
    Should trigger waitlist notification.
    """
    # TODO: Implement
    pass
```

```
def get_available_spots(self) -> int:
    """Return number of available spots."""
    # TODO: Implement
    pass
```

```
def calculate_revenue(self) -> float:
    """Calculate total revenue from ticket sales."""
    # TODO: Sum all ticket prices
    pass
```

```
def get_vip_count(self) -> int:
    """Return count of VIP tickets sold."""
    # TODO: Implement for VIP limit checking
    pass
```

```
class Conference(Event):
```

```
    """
    Multi-track conference with keynote speaker.
    Minimum 50 attendees required.
    """
```

```
    MIN_ATTENDEES = 50
```

```
    def __init__(self, event_id: str, name: str, event_date: date, capacity: int,
                  num_tracks: int, keynote_speaker: str, base_ticket_price: float = 200.0):
        # TODO: Implement
        pass
```

```
    def display_details(self) -> None:
        # TODO: Show conference details including tracks and keynote
        pass
```

```
    def is_registration_open(self) -> bool:
        # TODO: Check capacity and date
        pass
```

```
class Workshop(Event):
```

```
    """
    Hands-on workshop with limited capacity.
    Maximum 30 participants.
    """
```

```
    MAX_CAPACITY = 30
```

```

def __init__(self, event_id: str, name: str, event_date: date, capacity: int,
              instructor: str, skill_level: str, materials_included: bool,
              base_ticket_price: float = 150.0):
    # TODO: Implement with capacity validation
    pass

def display_details(self) -> None:
    # TODO: Show workshop details
    pass

def is_registration_open(self) -> bool:
    # TODO: Check capacity (max 30)
    pass

```

```

class Webinar(Event):
    """
    Online event with unlimited capacity.
    Set capacity to -1 for unlimited.
    """

    def __init__(self, event_id: str, name: str, event_date: date,
                  platform: str, recording_available: bool, timezone: str,
                  base_ticket_price: float = 50.0):
        # TODO: Implement with unlimited capacity
        pass

    def display_details(self) -> None:
        # TODO: Show webinar details including platform and timezone
        pass

    def is_registration_open(self) -> bool:
        # TODO: Always open if before event date (unlimited capacity)
        pass

    def get_available_spots(self) -> int:
        # TODO: Return -1 for unlimited
        pass

```

```

# =====
# SESSION MANAGEMENT
# =====

```

```

class Session(Schedulable):
    """
    Represents a single session within an event.

    Attributes:
        session_id (str): Unique identifier
        title (str): Session title
        speaker (SpeakerAttendee): The presenter
        duration_minutes (int): Length of session
    """

```

```

        room (str): Room assignment
        start_time (datetime): When session starts
    """

def __init__(self, session_id: str, title: str, speaker: SpeakerAttendee,
              duration_minutes: int, room: str, start_time: datetime):
    # TODO: Implement
    pass

def get_time_slot(self) -> tuple:
    """Return (start_time, end_time) as datetime objects."""
    # TODO: Calculate end time from duration
    pass

def conflicts_with(self, other: 'Session') -> bool:
    """Check if this session conflicts with another."""
    # TODO: Check room and time overlap
    pass

def has_speaker_conflict(self, other: 'Session') -> bool:
    """Check if same speaker has overlapping sessions."""
    # TODO: Implement
    pass

class SessionScheduler:
    """
    Manages session scheduling and conflict detection.
    """

    def __init__(self):
        self._sessions: List[Session] = []

    def add_session(self, session: Session) -> None:
        """
        Add a session to the schedule.

        Raises:
            RoomConflictError: If room is already booked
            SpeakerConflictError: If speaker has conflicting session
        """
        # TODO: Implement with conflict checking
        pass

    def get_schedule(self) -> List[Session]:
        """Return all scheduled sessions sorted by start time."""
        # TODO: Implement
        pass

# =====
# FACTORY PATTERN
# =====

```

```

class EventFactory:
    """
    Factory for creating different types of events.

    Usage:
        factory = EventFactory()
        event = factory.create_event("conference", event_id="CONF001", ...)
    """

    def create_event(self, event_type: str, **kwargs) -> Event:
        """
        Create an event of the specified type.

        Args:
            event_type: One of "conference", "workshop", "webinar"
            **kwargs: Event-specific attributes

        Returns:
            Event: The created event instance

        Raises:
            ValueError: If event_type is unknown
            InvalidEventConfigError: If required kwargs are missing
        """
        # TODO: Implement factory logic
        pass

```

```

class TicketFactory:
    """Factory for creating different types of tickets."""

    def create_ticket(self, ticket_type: str, ticket_id: str,
                     event: Event, **kwargs) -> Ticket:
        """
        Create a ticket of the specified type.

        Args:
            ticket_type: One of "standard", "vip", "earlybird", "group"
            ticket_id: Unique ticket identifier
            event: The event this ticket is for
            **kwargs: Ticket-specific attributes

        Returns:
            Ticket: The created ticket instance
        """
        # TODO: Implement factory logic
        pass

```

```

# =====
# ADAPTER PATTERN
# =====
class LegacyTicketingSystem:

```

```

"""
Legacy ticketing system from previous vendor.
DO NOT MODIFY THIS CLASS.
"""

def __init__(self):
    self.old_bookings = {
        "BK001": {"guest_name": "John Smith", "email_addr": "john@email.com",
                  "event_code": "EVT100", "tier": "GOLD", "amount_paid": 15000},
        "BK002": {"guest_name": "Jane Doe", "email_addr": "jane@email.com",
                  "event_code": "EVT100", "tier": "SILVER", "amount_paid": 8000},
        "BK003": {"guest_name": "Bob Wilson", "email_addr": "bob@email.com",
                  "event_code": "EVT200", "tier": "BRONZE", "amount_paid": 5000},
    }

def fetch_booking(self, booking_ref: str) -> Optional[Dict]:
    """Fetch booking data in legacy format. Returns None if not found."""
    return self.old_bookings.get(booking_ref, None)

class LegacyBookingAdapter:
    """
    Adapter to convert legacy booking data to current system format.

    Conversions:
    - amount_paid (cents) -> ticket_price (dollars)
    - tier GOLD -> VIPTicket, SILVER -> StandardTicket, BRONZE -> StandardTicket
    - guest_name -> name
    - email_addr -> email
    """

    TIER_MAPPING = {
        "GOLD": "vip",
        "SILVER": "standard",
        "BRONZE": "standard"
    }

def __init__(self, legacy_system: LegacyTicketingSystem):
    # TODO: Store reference to legacy system
    pass

def convert_booking(self, booking_ref: str, event: Event) -> tuple:
    """
    Convert legacy booking to (Attendee, Ticket) tuple.

    Raises:
        ValueError: If booking_ref not found
    """
    # TODO: Implement conversion logic
    pass

# =====

```



```

# SINGLETON PATTERN - Event Registry
# =====

class EventRegistryMeta(type):
    """Metaclass implementing Singleton pattern."""
    _instances = {}

    def __call__(cls, *args, **kwargs):
        # TODO: Implement singleton logic
        pass

class EventRegistry(metaclass=EventRegistryMeta):
    """
    Central registry for all events and attendees.
    Implemented as a Singleton.
    """

    def __init__(self):
        # TODO: Initialize storage for events and attendees
        pass

    def register_event(self, event: Event) -> None:
        """Register an event in the system."""
        # TODO: Implement
        pass

    def find_event(self, event_id: str) -> Optional[Event]:
        """Find an event by ID."""
        # TODO: Implement
        pass

    def get_all_events(self) -> List[Event]:
        """Return all registered events."""
        # TODO: Implement
        pass

# =====
# MAIN DEMONSTRATION
# =====

def main():
    """
    Main function to demonstrate all system capabilities.

    Demonstrate:
    1. Creating events using the factory
    2. Creating different ticket types
    3. Registering attendees (successful and failed)
    4. Waitlist functionality (Observer pattern)
    5. Session scheduling with conflict detection
    6. Legacy system integration via adapter
    7. Exception handling for various error conditions
    8. Singleton verification for EventRegistry
    """

```

```

"""

print("=" * 60)
print("EventPro Conference Management System")
print("=" * 60)

# -----
# 1. FACTORY PATTERN DEMO - Event Creation
# -----
print("\n--- Creating Events via Factory ---")
# TODO: Create events using EventFactory
# Example:
# factory = EventFactory()
# pycon = factory.create_event("conference", ...)

# -----
# 2. TICKET CREATION
# -----
print("\n--- Creating Tickets ---")
# TODO: Demonstrate different ticket types
# - StandardTicket
# - VIPTicket (show 20% limit)
# - EarlyBirdTicket (show 30-day rule)
# - GroupTicket (show minimum 5 rule)

# -----
# 3. ATTENDEE REGISTRATION
# -----
print("\n--- Registering Attendees ---")
# TODO: Register various attendee types
# - Regular Attendee
# - SpeakerAttendee
# - SponsorAttendee

# -----
# 4. OBSERVER PATTERN - Waitlist Demo
# -----
print("\n--- Waitlist Demonstration ---")
# TODO: Fill event to capacity, add to waitlist
# Then cancel a registration and show notification

# -----
# 5. SESSION SCHEDULING
# -----
print("\n--- Session Scheduling ---")
# TODO: Schedule sessions
# Demonstrate conflict detection (room and speaker)

# -----
# 6. ADAPTER PATTERN DEMO
# -----
print("\n--- Legacy System Integration ---")
# TODO: Use LegacyBookingAdapter to import old bookings

```

```

# -----
# 7. EXCEPTION HANDLING DEMO
# -----
print("\n--- Exception Handling Demo ---")
# TODO: Demonstrate various exceptions:
# - EventFullError
# - DuplicateRegistrationError
# - InvalidTicketError
# - RoomConflictError
# - SpeakerConflictError

# -----
# 8. REVENUE REPORTING
# -----
print("\n--- Revenue Report ---")
# TODO: Calculate and display event revenue

# -----
# 9. SINGLETON VERIFICATION
# -----
print("\n--- Singleton Verification ---")
# TODO: Verify EventRegistry is a singleton
# reg1 = EventRegistry()
# reg2 = EventRegistry()
# print(f"Same instance? {reg1 is reg2}") # Should be True

if __name__ == "__main__":
    main()

```

Sample Expected Output

Your program should demonstrate functionality similar to:

```

=====
EventPro Conference Management System
=====

--- Creating Events via Factory ---
Created: Conference[CONF001] PyCon AU 2025 - Capacity: 500, Tracks: 4
    Keynote Speaker: Guido van Rossum
Created: Workshop[WS001] Django Masterclass - Capacity: 25
    Instructor: Sarah Chen | Level: Intermediate | Materials: Yes
Created: Webinar[WEB001] Intro to FastAPI - Capacity: Unlimited
    Platform: Zoom | Recording: Yes | Timezone: AEST

--- Creating Tickets ---
StandardTicket[T001] for CONF001: $200.00
    Benefits: Access to main sessions
VIPTicket[T002] for CONF001: $500.00
    Benefits: Priority seating, Networking dinner, Speaker meet-and-greet

```

```
EarlyBirdTicket[T003] for CONF001: $140.00 (30% off!)
  Benefits: 30% early bird discount
GroupTicket[T004] for CONF001 (10 people): $1,600.00 ($160.00/person)
  Benefits: 20% group discount

--- Registering Attendees ---
Registered: Alice Johnson (alice@email.com) with StandardTicket
Registered: Dr. James Wilson [SPEAKER] with complimentary VIPTicket
  Session: "Advanced Python Patterns"
Registered: TechCorp Representative (sponsor@techcorp.com) [SPONSOR]
  Company: TechCorp | Tier: Gold

--- Waitlist Demonstration ---
Event CONF001 is now full (500/500)
Bob Smith added to waitlist for CONF001 (Position: 1)
Carol White added to waitlist for CONF001 (Position: 2)

Cancelling registration for Alice Johnson...
  NOTIFICATION to Bob Smith: A spot is now available for PyCon AU 2025!
Bob Smith automatically registered from waitlist.

--- Session Scheduling ---
Scheduling sessions for CONF001...
✓ Session[S001] "Keynote: Future of Python" scheduled in Main Hall at 09:00
✓ Session[S002] "Advanced Python Patterns" scheduled in Room A at 10:30
x RoomConflictError: Room A is already booked at 10:30-11:30
x SpeakerConflictError: Dr. James Wilson already presenting at 10:30

--- Legacy System Integration ---
Converting legacy booking BK001...
Converted: John Smith (john@email.com) with VIPTicket ($150.00)
Converting legacy booking BK002...
Converted: Jane Doe (jane@email.com) with StandardTicket ($80.00)

--- Exception Handling Demo ---
Attempting to register duplicate email...
x DuplicateRegistrationError: alice@email.com is already registered for CONF001

Attempting to purchase EarlyBirdTicket 5 days before event...
x InvalidTicketError: EarlyBird tickets require 30+ days before event

Attempting to create GroupTicket with 3 people...
x InvalidTicketError: Group tickets require minimum 5 attendees

--- Revenue Report ---
Event: PyCon AU 2025
  Standard Tickets: 380 × $200.00 = $76,000.00
  VIP Tickets: 100 × $500.00 = $50,000.00
  Early Bird Tickets: 20 × $140.00 = $2,800.00
  Total Revenue: $128,800.00

--- Singleton Verification ---
Registry 1 ID: 140234567890
```

Marking Rubric

Criteria	Excellent (100%)	Good (75%)	Satisfactory (50%)	Needs Work (25%)	Marks
OOP Design & Architecture	Elegant design with clear structure and minimal redundancy	Good design with clear structure and some redundancy	Basic design work with some structure	Poor design; pattern misuse	/20
Encapsulation	All attributes properly encapsulated	Most attributes properly encapsulated	Some encapsulation present	Public attributes throughout	/10
Inheritance & Polymorphism	Deep, meaningful hierarchy	Good hierarchy with some abstraction	Basic inheritance structure	Flat structure or base class misuse	/15
Abstraction & Interfaces	Well-designed ABCs	ABCs present with some utility	Basic ABC implementation	Missing or non-functional	/10
Custom Exceptions	Complete hierarchy of exceptions	Good exception handling	Basic exceptions present	Few or no custom exceptions	/10
Design Patterns	All 4 patterns (Factory, Singleton, Observer, Strategy) correctly implemented	3 patterns correctly implemented	2 patterns correctly implemented	1 or fewer patterns implemented	/15
Code Quality	Clean, documented, and easy to read	Good quality with some comments	Functional but messy	Difficult to read or no comments	/5
AI Copilot Usage	Excellent documentation of AI interactions	Good documentation of AI interactions	Basic documentation of AI interactions	Poor/no documentation	/10
Reflection Document	Insightful rationale and analysis	Good rationale and analysis	Basic reflection	Missing or superficial	/5

Total: /100

Submission Requirements

Submit the following files:

- event_management_system.py Your complete Python implementation
- NIT2112_Reflection_[YourStudentID].pdf Containing:
 - Design Rationale (max 500 words): Explain your class design choices, hierarchy structure, and trade-offs
 - AI Interaction Log: Document at least 5 significant AI interactions:
 - The prompt you used
 - Summary of AI's response
 - How you used/modified/rejected the suggestion and why

Time Management Suggestion

Phase	Time	Activities
Analysis & Design	10 min	Read spec, sketch class diagram, as
Core Implementation	30 min	Implement classes, patterns, except
Testing & Refinement	15 min	Test edge cases, fix bugs, add docu
Reflection Writing	5 min	Complete design rationale and AI lo

Final Tips

- Start with Event and Ticket hierarchies They're the foundation

2. Test incrementally Don't write everything then test
3. Use AI strategically Ask focused questions, not "write everything"
4. Handle errors gracefully Use your custom exceptions throughout
5. Comment wisely Explain non-obvious design decisions

Academic Integrity

- The conceptual design and core problem-solving logic must be your own work, facilitated by AI
- Blindly copying large blocks of AI code without understanding or adaptation is academic malpractice
- Be prepared to explain any part of your code and design rationale during review

Good luck! This test assesses your ability to design and implement OOP solutions WITH AI assistance as a collaborative partner.